REPORT · 2026年8月

造像类文物三维建模自动化系统 publish代码说明书

M6 发布归档 · 交付包 · 溯源 · 主线 M1~M6 收官总结

文保数字化研发组
2026年8月

目 录

一、概述 3

二、模块划分 4

三、依赖关系与数据流 5

3.1 依赖方向 5

3.2 发布数据流 5

四、模块详细设计 6

4.1 export_delivery — 交付模型多格式导出 6

4.2 build_meta / write_meta — 元数据快照（溯源） 6

4.3 copy_qc_report — 质检报告归档 6

4.4 build_delivery_manifest — 批次交付清单 6

4.5 runner M6 阶段接入（STAGE_REGISTRY） 6

五、接口契约与错误码 7

六、测试与验证 8

七、主线收官总结（M1~M6 全链路完成） 9

7.1 全部子包完成度 9

7.2 后续方向（主线收官后） 9

附录 A：run_manifest.json 六阶段溯源示例 10

一、概述

本文档是《造像类文物三维建模自动化系统（路线 E）》**publish 子包（M6 发布与溯源，L2 业务层 / S4 阶段）**的代码说明书，同时是**主线收官总结。**publish 负责把 M4 带色模型导出为标准开放格式、嵌入元数据与 manifest 快照，形成可交付包（设计书 §37）：交付模型（80_export）+ 元数据快照（90_meta）+ 质检报告归档 + 交付清单。该子包接入 M0 编排内核（@register_stage("M6")）后，**「点云 -> 模型 -> 带色 -> 质检 -> 发布」主线 M1~M6 全链路正式收官。**

> 设计要点：**交付包 = 模型 + 质检报告 + manifest 一并归档**（软著方案 2.4）。M6 从 task 内存状态取各阶段指标生成元数据快照（不依赖磁盘 manifest——manifest 在批处理结束后才落盘，直接读会 FileNotFoundError，已在开发中修复）。任一质量门不达标时该尊已在 M5 被隔离，天然不进入交付包（设计书 §6.3）。

二、模块划分

```
src/h3d/publish/
+-- __init__.py      # 发布归档流水线（单文件子包）
    +-- export_delivery       # 50_tex L1 模型 -> 80_export/<item>/（多格式标准命名）
    +-- build_meta / write_meta # manifest 溯源元数据（90_meta/<item>_meta.json）
    +-- copy_qc_report        # 质检报告 accuracy.csv 归档进交付包
    +-- build_delivery_manifest # 交付清单（90_meta/delivery_manifest.json，验收依据）
```

函数	核心职责	对应设计书
export_delivery(tex_dir, out_dir, item, formats)	交付模型多格式导出（L1 优先，LOD 回退）	§37.2 export
build_meta / write_meta	manifest 快照元数据（项目 / 阶段指标 / 软件版本 / 配置哈希）	§37.2 embed_meta
copy_qc_report	质检报告 accuracy.csv 归档进交付包	软著方案 2.4 交付包
build_delivery_manifest	批次交付清单汇总（状态 / 模型 / 质检 / 哈希）	TC-602 元数据可解析

三、依赖关系与数据流

3.1 依赖方向

publish 属 L2 业务层，依赖 core 底座（metadata 溯源：软件版本 / run_id / 配置哈希）与 io 产物，不反向；M6 阶段从 task 内存结果取指标（与磁盘 manifest 解耦）：

```
package-level deps:
  publish --> core.metadata (software_versions / generate_run_id / sha256_text)
  publish --> 标准库 (shutil / json / os)
  runner M6 --> h3d.publish + task.results（内存指标，非磁盘 manifest）
  core / io / preprocess / register / mesh / texture / quality / change / colour
          --> (NONE; 不反向依赖 publish)
```

3.2 发布数据流



M6 发布数据流：交付模型导出 → 元数据快照 → 质检报告归档 → 交付清单 → 可交付包

四、模块详细设计

4.1 export_delivery — 交付模型多格式导出

交付模型取自 M4 最高精度带色模型（50_tex/<item>_L1.<fmt>），复制到 80_export/<item>/ 并去掉 LOD 标签（标准命名 <item>.<fmt>，TC-603 多格式均生成）。L1 缺失时回退任意 LOD 变体并告警；某格式完全缺失时跳过（E601 语义）；目标不可写抛 ExportError。

4.2 build_meta / write_meta — 元数据快照（溯源）

元数据快照 = 项目信息 + 该尊各阶段指标（M1~M5 从 task.results 内存取）+ 软件版本 + run_id + 配置哈希（sha256(config)），写入 90_meta/<item>_meta.json——构成「每一份模型可回溯到原始数据、处理参数与软件版本」的溯源依据（设计书 §6.3 / §37.2，TC-602）。

4.3 copy_qc_report — 质检报告归档

把 70_report/<item>/accuracy.csv 复制进 80_export/<item>/（copy_qc=true），交付包自带验收依据；无质检报告时记日志跳过（不阻塞发布）。

4.4 build_delivery_manifest — 批次交付清单

90_meta/delivery_manifest.json 汇总本批次全部交付包（item_id / status / models / qc_status / meta / config_hash），重跑时追加合并（断点续跑语义一致）。

4.5 runner M6 阶段接入（STAGE_REGISTRY）

```
@register_stage('M6')
def _stage_publish(task, cfg):
    tex_dir = f'{task.root}/50_tex/{task.item_id}'
    models = export_delivery(tex_dir, f'{task.root}/80_export/{task.item_id}',
                             task.item_id, cfg.publish.formats)
    # 各阶段指标从 task 内存取（manifest 批处理结束才落盘）
    stage_metrics = {sid: dict(task.results[sid].metrics) for sid in M1..M5}
    meta = build_meta(item_id, stage_metrics, project, software, run_id, hash)
    meta_path = write_meta(meta, f'{task.root}/90_meta', item_id)
    if cfg.publish.copy_qc:
        qc_report = copy_qc_report(task.root, export_dir, item_id)
    delivery_manifest = build_delivery_manifest(entries, f'{task.root}/90_meta/...')
    return StageResult('M6', DONE, outputs, metrics)
```

配置项	默认值	用途
publish.enabled	true	M6 发布开关
publish.formats	[ply, obj, glb]	交付模型标准开放格式
publish.embed_meta	true	是否生成 manifest 快照元数据（溯源）
publish.copy_qc	true	是否把质检报告归档进交付包

五、接口契约与错误码

错误码	异常类	触发场景	处理策略
E100	InputDataError	M6 缺少 M4 带色模型目录	失败隔离，记录后跳过
E601	ExportError	交付模型复制失败（目标不可写 / 路径错误）	失败隔离，记录后跳过
（跳过）	—	某格式模型缺失	跳过该格式 + 告警（设计书 §37.4）
（降级）	—	元数据不完整	仍导出模型，标记 meta 不全

六、测试与验证

publish 配套 10 项单元测试 + 端到端 1 项新增（M6 收官），全量 **232 项** 通过：29 core + 20 io + 16 preprocess + 11 register + 21 mesh + 20 texture + 22 quality + 21 report + 31 runner + 6 cli + 10 change + 14 colour + 10 publish + 1 端到端新增。

测试文件	覆盖点
tests/test_publish/test_publish.py	多格式导出 / LOD 回退 / 缺失格式跳过 / 带色保留 / 元数据快照 / 交付清单追加 / accuracy.csv 归档
tests/test_runner/test_pipeline.py::test_m6_publish_full_closure	M1~M6 全链路、80_export 交付模型、90_meta 元数据 + 交付清单、manifest M6 指标

```
$ python -m pytest -q
..................................................................... [ 33%]
..................................................................... [ 66%]
..................................................................... [100%]
232 passed, 8 warnings in 38.74s

$ python -m h3d.cli batch -c demo.yaml
M6 完成：rock001（3 格式模型 + 元数据快照 + 交付清单）
M6 完成：rock002（3 格式模型 + 元数据快照 + 交付清单）
批量处理完成：total=2 done=2 failed=0 skipped=0

交付包结构（80_export + 90_meta）：
  80_export/rock001/{rock001.ply, rock001.obj, rock001.glb, accuracy.csv}
  90_meta/{rock001_meta.json, rock002_meta.json, delivery_manifest.json, run_manifest.json}
  交付清单: n_items=2 n_published=2（qc=done, models=3/尊）
```

七、主线收官总结（M1~M6 全链路完成）

7.1 全部子包完成度

层级 / 子包	状态	测试覆盖
L0 core 底座 / L1 io	已交付	49
L2 preprocess(M1) / register(M2) / mesh(M3) / texture(M4) / quality(M5)	已交付	90
L2 publish（M6 发布归档）	**本次交付**	**11**
L2 change（X1 M3C2）/ colour（X2）	已交付	24
L2 report（M6 报表）	已交付	21
L3 runner（M0 编排）+ L4 cli	已交付	37

「点云 -> 模型 -> 带色 -> 质检 -> 发布」主线 M1~M6 全链路正式收官，可选模块 X1/X2 亦已交付。STAGE_REGISTRY 六阶段（M1 预处理 → M2 配准 → M3 重建 → M4 纹理 → M5 质检 → M6 发布）一键跑通，交付包（80_export + 90_meta + run_manifest）完整可审计。测试 232 项 100% 通过；许可证全部 MIT/BSD；全部模块遵循「开源、可复现、可量化」原则（设计书 §1）。

7.2 后续方向（主线收官后）

优先级	方向	目标	关键点
P1	X1/X2 接入编排	change/colour 条件注册为可选 stage（enabled 时执行）	X1 需约定 t1/t2 目录规范
P2	CloudCompare 交叉校验实装	change.validation 完整调用 CC CLI M3C2	外部可执行文件依赖
P2	并发调度 / HTML 报表	ProcessPoolExecutor + jinja2	与工程化打磨合并排期
P3	UV 图集增强	M4 texture paint_mode='uv' 实装（xatlas）	python-xatlas MIT 依赖

> **收官结论**：主线（M0 编排 + M1~M6）+ 可选（X1/X2）已全部交付，232 项测试 100% 通过，交付包可审计可溯源。系统达到「批处理、可复现、可溯源」的初始设计目标（设计书 §6.3）。

附录 A：run_manifest.json 六阶段溯源示例

```
run_manifest.json 阶段条目（rock001/M1 ~ M6）：
  rock001/M1: {status: done, n_cleaned: 2, n_removed: 43, ...}
  rock001/M2: {status: done, registered: true, fitness: 0.98, ...}
  rock001/M3: {status: done, n_vertices: 13272, n_triangles: 26400, ...}
  rock001/M4: {status: done, colored: true, paint_mode: vertex, n_outputs: 6, ...}
  rock001/M5: {status: done, coverage: 0.694, rmse_mm: 219.3, mps_m: 0.048, ...}
  rock001/M6: {status: done, formats: [ply, obj, glb], n_models: 3,
               meta: rock001_meta.json, delivery_manifest: delivery_manifest.json}

90_meta/rock001_meta.json 溯源快照：
  {item_id, run_id, config_hash, project, software{open3d, numpy, ...},
   stages{M1..M6}, M6_outputs[...], delivery{models, qc}}
```